Week 2 Tutorial

Welcome to Week 2 Tutorial. Please follow these instructions step by step - your tutors will also take you through these one by one in class. Please ask for their assistance if you're stuck at any place - we are here to help. Today, you will learn the following:

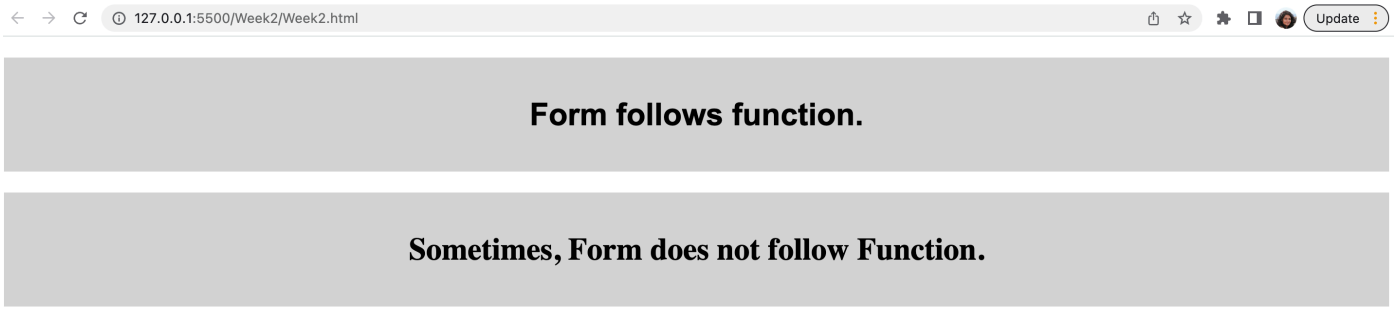
- Lecture 2 review
- Some more HTML and SVG: Classes and IDs
- Comments and Clarity
- Understanding RGB
- Drawing SVG with JS:
  - New method: `document.getElementbyId()`
  - New method: `Element.setAttribute()`
  - New method: `Math.random()`
  - New method: `Math.round()`
  - New method: `appendChild()`

1: Lecture 2 Review

Carefully read through and review Lecture 2. Do all the steps yourself. Ask your tutors if something is not clear, or you're stuck somewhere. Practicing and becoming confident with these basics will make you a confident programmer.

2: Some more HTML and SVG: Classes and IDs

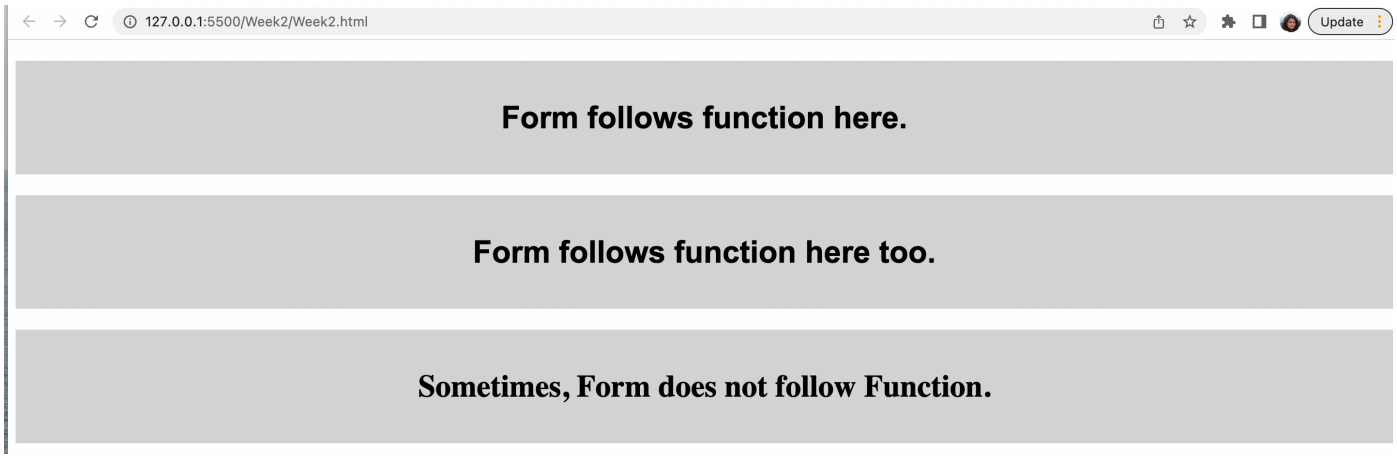
The HTML `id` attribute specifies a unique id or name for an HTML element. Referring to this name in other parts of the code, like CSS or JS, enables us to style the element, or change its structure or behaviour. Remember that if you use ids, each id is unique, and can be assigned to exactly one element. For example,



As you can see, I may have a CSS style that applies to all `<h1>` headings, but I may give a unique id to just one of them and apply a style change (referring to an id in CSS is done by using the # symbol followed by the name of the id). In this case, changing the standard Times New Roman serif font to the more modern sans-serif family.

An id can be assigned to both HTML as well as SVG elements. We will see how to use this later in the tutorial, but for now just remember that I can name my SVG elements in the same way as I name my HTML elements, using the id attribute.

If I want a certain set of rules to apply to a bunch of elements, I can use the `class` attribute. Assigning a class name to multiple elements means I want all the elements in that class to look or behave in the same way. In the CSS, a class is written as a `.` followed by the class name. For example:



As you can see, two `<h1>` elements here shared the same class, and the CSS rule specified for that class applied to them, but not to another `<h1>` element which doesn't belong to that class. The main difference between an id and a class is that an id is applied to a single element and is a unique identifier for that element, whereas a class can be applied to a sub-set or group of elements.

A list of all CSS selectors is here: **https://www.w3schools.com/css/css_selectors.asp (https://www.w3schools.com/css/css_selectors.asp)**

3: Comments and clarity

You may have already discussed this in your last session with your tutor, but let's review code comments and briefly touch upon the importance of clarity.

Comments are pieces of text within our code that are ignored by the computer but are readable by humans. It's crucial to comment our code effectively so that others (or ourselves at a later date) can understand what it does clearly. Reading code you wrote a week, a month, or even a year earlier without comments can be incredibly confusing.

As we are using JavaScript and HTML in this course, it's important to understand how to write comments in both. Let's take a closer look:

HTML

Our HTML files might often be quite straightforward now that we're doing so much with JavaScript. It's not necessary to comment on required HTML code, but you can add comments for clarity if it's appropriate. Here's how you can add comments in HTML:

```
<!-- This is a comment in HTML -->
```

Comments in HTML are enclosed in `<!--` and `-->`. Anything between these will be ignored by the browser, making it a perfect place to add notes or explain parts of your HTML code.

Javascript

In JavaScript, commenting your code well is essential to understand what it's doing. In JavaScript, we have single-line and multi-line comments. They work like this:

- **Single-line comments** are created using two forward slashes (`//`). Anything following `//` on the same line will be considered a comment and ignored by JavaScript.

```
// This is a single-line comment in JavaScript
```

- **Multi-line comments** start with `/*` and end with `*/`. Everything between these markers will be treated as a comment, regardless of how many lines it spans.

```
/*
This is a multi-line comment in JavaScript
which spans over multiple lines.
*/
```

Camel Case

A common convention in JavaScript for naming variables is to use *camelCase*. In camelCase, the first word is lowercase, and each subsequent word starts with an uppercase letter, like this:

```
let myVariableName; // Correct camelCase example
let userAge; // Another camelCase example
```

Using camelCase for variable names improves readability and is a widely accepted practice in the JavaScript community.

Variable names

When it comes to variable names, we've learned that we can name our variables anything we want (with the exception of **reserved keywords (https://www.w3schools.com/js/js_reserved.asp)**). However, it's essential to write our code as if a complete stranger will read it. This means choosing clear and descriptive variable names. Even though we can name a variable anything, we should avoid being vague or using names like `myNumber` , `newNumber` , or similar. Instead use descriptive names like userAge or totalSVGWidth.

Check the quiz descriptions, they will now require good and useful comments and camel case and sensible names for variables.

4: Understanding RGB and Hex Codes

Please head over to **http://www.cknuckles.com/rgbsliders.html (http://www.cknuckles.com/rgbsliders.html)** and explore the tool there. Basically, all colours we use

are defined by using the RGB (R=Red, G=Green, B=Blue) scheme. Each colour can vary from a minimum value of 0 to a maximum value of 255, 256 numbers in total = 2^8. The tutors will demonstrate to you on the tool how different colours are produced by using different quantities of R, G, and B components.

RGB colours can be expressed using the `rgb(0, 0, 0)` format. The rgb format simply chooses a number between `0` and `255` (ends inclusive) to mix red, green, and blue in different proportions. For example `rgb(0, 0, 0)` means black, `rgb(255, 255, 255)` means white, and `rgb(255, 0, 0)` means red. Other colours can be made by mixing these. For example, pure blue and green may be mixed to produce cyan, and this is represented by `rgb(0, 255, 255)`.

A second way to define colours is using HEX codes, which use hexadecimal numbers. In essence, instead of the usual base 10 decimal number system, hexadecimal numbers are base 16 numbers:

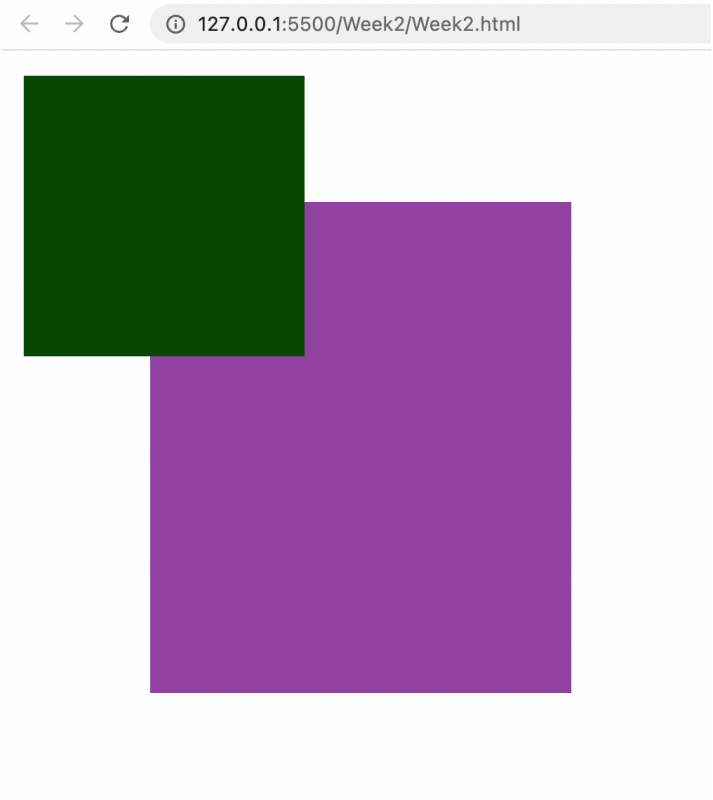
HEX number	Decimal equivalent
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
A	10
B	11
C	12
D	13
E	14
F	15

Thus, you have 16 total possibilities - from 0 to F, to represent numbers from 0 to 15. HEX codes represent colours by a hash followed by 6 positions - 2 for red, 2 for green, 2 for blue. For example, `#000000` means black and `#FFFFFF` means white. With this logic, since cyan is blue plus green, the hex code will be `#00FFFF`. Once you understand this basic concept, you can head over to **https://color.adobe.com/Find-Hex-Codes-color-theme-7154985/**

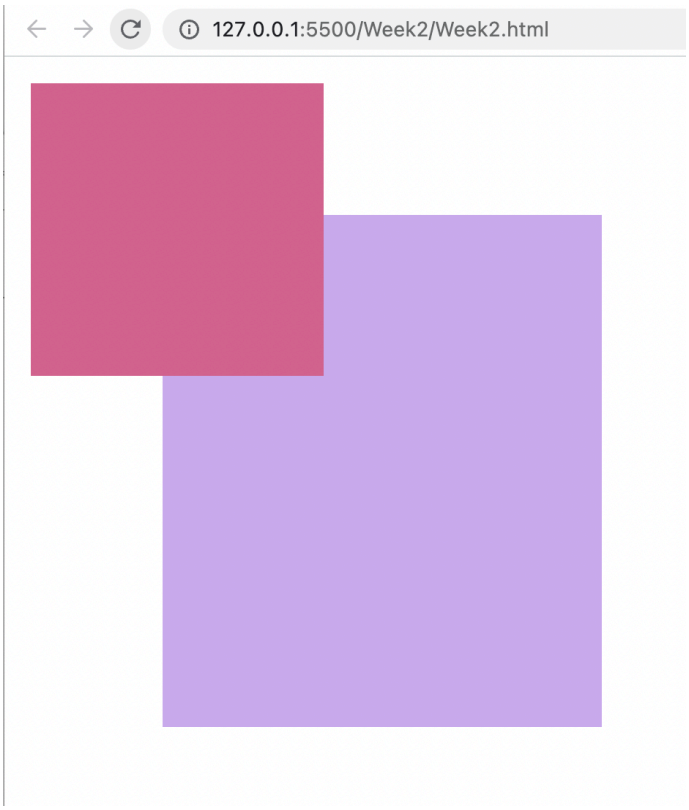
(https://color.adobe.com/Find-Hex-Codes-color-theme-7154985/) - this is a very handy tool when you're choosing colours for your projects.

5: Drawing SVG with JS

We now have enough knowledge to start learning some basic functions, so that we can actually start to do some interesting things. By the end of the tutorial, here is what you would have achieved: we have generated two squares with different random fills in an SVG...



Then, upon refreshing the page, each time you have new random colours:



This may look simple, but we have not coded this using direct SVG into the HTML document. Instead, we have used JavaScript to dynamically generate content for our HTML page. Later when we get more into interaction design, we will enable the same behaviour using events like clicking or hovering over elements. Further, there are many more efficient ways of writing this code, using libraries like d3.js or p5.js - but at the moment we want to concentrate on learning the basics of the language.

Right now, let's have a look at the full code which enables this, and then we will break it apart part by part:

THE HTML CODE: Week2.html

```
<!DOCTYPE html>
<html>
    <head>
        <title>IDEA9103, Week 1</title>
        <meta charset="UTF-8" />
        <link rel="stylesheet" href="Week2.css" />
    </head>
    <body>
        <svg id="base_svg" width="600" height="500">
        <rect id="myRectangle" x="100" y="100" width="300" height="100"></rect>
        </svg>
        <script type="text/javascript" src="script.js"></script>
    </body>
</html>
```

The tutors will explain this code bit by bit, but you should already understand all of it. Note how the svg element and the rect elements each have an id.

THE JAVASCRIPT CODE: script.js

```
let rectangle = document.getElementById('myRectangle');
console.log(rectangle);

rectangle.setAttribute("height", "350");

let r = Math.round(Math.random() * 255);
let g = Math.round(Math.random() * 255);
let b = Math.round(Math.random() * 255);
rectangle.setAttribute('fill', `rgb(${r}, ${g}, ${b})`);

const svg = document.getElementById("base_svg");
console.log(svg);

let newRectangle = document.createElementNS("http://www.w3.org/2000/svg", "rect");
newRectangle.setAttribute("x", "10");
newRectangle.setAttribute("y", "10");
newRectangle.setAttribute("width", "200");
newRectangle.setAttribute("height", "200");
let r1 = Math.round(Math.random() * 255);
let g1 = Math.round(Math.random() * 255);
let b1 = Math.round(Math.random() * 255);
newRectangle.setAttribute('fill', `rgb(${r1}, ${g1}, ${b1})`);
svg.appendChild(newRectangle);
```

This first bit of code declares a variable called `rectangle`, into which it "fetches" the rectangle already created on your HTML page. The word `document` refers to the root HTML document. Then, the dot (.) notation adds a method to this document object called `getElementById()`. The `getElementById()` method literally GETS an element with the given id. In this case, in the HTML code we declared a rectangle element with the id x. Thus, we fetch this particular rectangle element, and put it inside the variable `rectangle`. Methods like `getElementById()` and `querySelector()` allow you to choose existing elements on the HTML page and dynamically change their properties. In this example, if I remove the javascript file reference from the HTML page, and run the HTML in a browser, what I see is that a black rectangle is generated on the page with the given dimensions and position.

5.2 New method: `Element.setAttribute()`

```
rectangle.setAttribute("height", "350");
```

In the next line, you use the `setAttribute()` method to change the properties of the rectangle in any way you want. For example, here, the `"height"` property has been changed to `"350"` from the original `"500"` in the HTML page. The main idea to capture here is how you are using a JavaScript method to change the property of an HTML element dynamically during run time. In the same way, you can change any other property, like x, y, width, fill, or stroke. Try some of these options with your tutors guiding you.

5.3 New methods: `Math.round()` and `Math.random()`

Now we want to change the rectangle's colour. In fact, each time the page loads, we want to randomise the colour, with a new colour being chosen each time. So, we write:

```
let r = Math.round(Math.random() * 255);
let g = Math.round(Math.random() * 255);
let b = Math.round(Math.random() * 255);
rectangle.setAttribute('fill', `rgb(${r}, ${g}, ${b})`);
```

Here, we are declaring `r`, `g`, and `b` to be variables. Using **`Math.random()`** (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/random) will generate a floating point random number between 0 and 1, and multiplying it by 255 will scale this random number to a number between 0 and 255. So, we have three random `r`, `g`, and `b` values. Then, we use the **`Math.round()` method** (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/round) to round a floating point number to the nearest Integer - since we need integer `r`, `g`, and `b` values. Notice how one method is nested inside another method, so that the output from one method is a straight input into another method.

Then, we use the `setAttribute()` method again to change the `'fill'` property by assigning it these new `r`, `g`, and `b` values. Let's unpack this code, because its a bit complex:

To create the value for the fill property, we are using a **string template** (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Text_formatting#multi-line_template_literals): `` `rgb(${r}, ${g}, ${b})` ``

Templates are a programming technique in which a string is written with placeholder values, and real values replace the temporary placeholders. If you've ever made a mail merge document, you've already used this technique!

The template is different to a normal string because it is enclosed in backticks , e.g.: `...`, and you can use the special ${...} pattern to state where in the string a value should go. The template is evaluated and turned into a string immediately, this makes it a useful shortcut for creating structured text, like our rgb(r, g, b) property. There are more tricks that JavaScript's templates can do, but this is all we need to know for now.

Finally to step through the template we're using in our code, let us assumer that we have our three variables:

```
let r = 11;
let g = 22;
let b = 133;
```

If we assign our template to a variable:

`let rgbString = ` `` `rgb(${r}, ${g}, ${b})`; ``

The variable will now hold the string: "`rgb(11, 22, 133)`"

5.4 New method: `createElementNS()`

Up to this point, we have learnt how to fetch an existing rectangle from the HTML document and change its properties. We can also add new elements to the HTML page - those that did not exist before. Let's add a new rectangle.

You declare a constant called `svg` and use the `getElementById()` function to get the existing svg element by its id.

```
const svg = document.getElementById("base_svg");
console.log(svg);
```

```
let newRectangle = document.createElementNS("http://www.w3.org/2000/svg", "rect");
```

Now declare a new variable called `newRectangle` and use the `createElementNS()` function to create a new svg element. The first input "`http://www.w3.org/2000/svg`" specifies that the namespace NS is svg and thus we are creating an svg element, the second input "`rect`" signifies that this element is of type rect.

We have now created a new svg element, but it won't be visible in the page yet. As before, we set some attributes:

```
newRectangle.setAttribute("x", "10");
newRectangle.setAttribute("y", "10");
newRectangle.setAttribute("width", "200");
newRectangle.setAttribute("height", "200");
let r1 = Math.round(Math.random() * 255);
let g1 = Math.round(Math.random() * 255);
let b1 = Math.round(Math.random() * 255);
newRectangle.setAttribute('fill', `rgb(${r1}, ${g1}, ${b1})`);
```

5.5 New method: `appendChild()`

Finally, we have created this new rect, but we need to add it as a "child" to the existing `svg` element - which is being represented in the `svg` const we have defined. So, we use the `appendChild()` method to add this `newRectangle` to the svg.
`svg.appendChild(newRectangle);`

Once you have this code running successfully, try refreshing the page a few times to see the colours change randomly. Remember, none of this is hardcoded into the HTML—it's all happening

dynamically at runtime through your JavaScript code. Once you're comfortable with this, challenge yourself by creating your own generative artwork. Use JavaScript to dynamically modify the properties of shapes, applying the new methods you've learned today.

Happy coding!